

Topic 6: Neural Networks

Instructor: Dr. Akash Gupta

Table of contents

1	What is a Neural Network?	3
2	Start Here: A Single Neuron	3
2.1	What a Single Neuron Does	3
2.2	A Tiny Worked Example	4
3	Activation Functions: What “Bends” the Line	5
4	From One Neuron to One Layer	6
5	Stacking Layers: The Full Network	6
5.1	How a Prediction Flows Through the Network	6
6	Training: How the Network Learns from Mistakes	7
6.1	What is the “Loss”?	8
6.2	The Intuition Behind Backpropagation	8
7	The Adam Optimizer	9
8	Regularization: Keeping the Network Honest	9
8.1	L2 Regularization (Weight Decay)	9
8.2	Dropout	10
8.3	Early Stopping	10
9	Building a Neural Network in Python	10
10	Choosing Your Architecture	11
11	Advantages and Limitations	12
11.1	Advantages	12
11.2	Limitations	13
12	From Neural Networks to Large Language Models	13
12.1	The Neural Network Family Tree	13
12.2	How Transformers Work: Four Key Ideas	14
12.3	Scale Changes Everything	15

12.4What LLMs Are and Aren't	16
12.5Business Applications of LLMs	16
13 Summary	17

1 What is a Neural Network?

A **neural network** is a predictive model made of many simple computing units called **neurons**, wired together so they can learn patterns from data. Despite the biological-sounding name, a neural network is nothing mystical. At its heart it is a machine that takes numbers in, passes them through layers of neurons, and produces a prediction out the other side.

Neural networks power most of modern AI: image recognition, voice assistants, recommender systems, and large language models like ChatGPT and Claude.

Key Takeaway — Why Neural Networks Matter

- **Flexible:** Can learn almost any pattern, including curved and nonlinear relationships
- **Scalable:** Keep getting better as you feed them more data
- **Versatile:** Work on images, text, audio, video, and structured business data
- **The engine behind modern AI:** ChatGPT, facial recognition, and self-driving cars all use neural networks

Tradeoff: Harder to interpret than decision trees or linear models, and they need more data and tuning to shine on simple tabular problems.

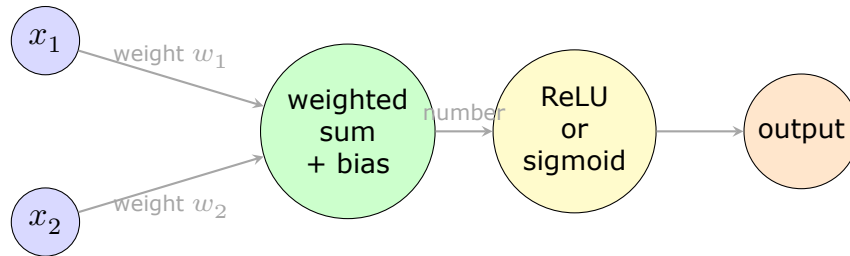
2 Start Here: A Single Neuron

Forget the full network for a moment. The smallest unit is a **single neuron**. Understand this one piece first, and everything else is just “many neurons working together.”

2.1 What a Single Neuron Does

A neuron takes in numbers, combines them, and produces one number. That is it. Here is what happens inside, step by step:

- **Step 1 — Take inputs.** A neuron receives one or more input numbers (for example, customer age, income, tenure)
- **Step 2 — Weight each input.** Each input is multiplied by a **weight**. The weights are what the network “learns” during training
- **Step 3 — Add everything up, plus a bias.** All the weighted inputs are summed together, and a constant called the **bias** is added on top
- **Step 4 — Pass through an activation function.** The sum goes through a simple function (like ReLU or sigmoid) that decides the neuron’s final output
- **Step 5 — Output one number.** That one number is what the neuron passes forward to the next neurons in the network



2.2 A Tiny Worked Example

Let us walk through one single neuron with two inputs. Suppose the neuron has:

- Weight on input 1: **0.6**
- Weight on input 2: **-\$0.4**
- Bias: **0.2**
- Activation function: **ReLU** (turns any negative number into 0, leaves positive numbers alone)

2.2.1 Case 1: Inputs (3, 2)

- Weighted sum: $(0.6 \times 3) + (-0.4 \times 2) + 0.2 = 1.8 - 0.8 + 0.2 = 1.2$
- ReLU of 1.2 is just 1.2 (positive, so it passes through)
- **Neuron output: 1.2** — the neuron “fires”

2.2.2 Case 2: Inputs (1, 4)

- Weighted sum: $(0.6 \times 1) + (-0.4 \times 4) + 0.2 = 0.6 - 1.6 + 0.2 = -0.8$
- ReLU of -0.8 is 0 (negative, so ReLU zeroes it out)
- **Neuron output: 0** — the neuron stays “silent”

Key Takeaway — The Intuition

A neuron is basically a little decision-maker. It looks at some inputs, weighs which ones matter more, adds them up, and then decides “do I fire or stay quiet?” The weights control what the neuron pays attention to, and training adjusts those weights so the neuron fires for the right patterns.

Under the Hood — Optional Formal Notation

Students who want the algebra: for a neuron with inputs $x_1, x_2, \dots, x_p$, weights $w_1, w_2, \dots, w_p$, bias b , and activation g :

$$z = w_1x_1 + w_2x_2 + \dots + w_px_p + b, \quad \text{output} = g(z)$$

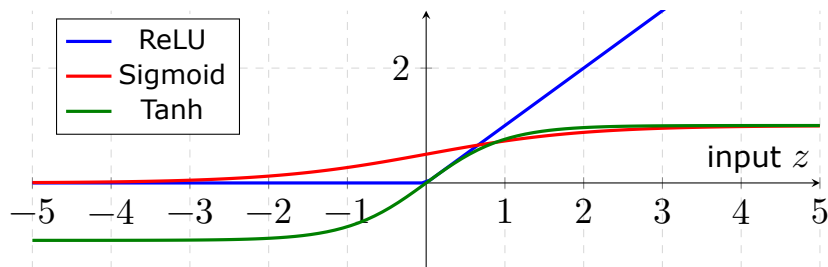
You can safely skip this box — the plain-English version above is all you need to follow the rest of the topic.

3 Activation Functions: What “Bends” the Line

Without an activation function, a neuron is just a weighted sum — a straight-line combination of its inputs. Straight lines can only learn simple patterns. **Activation functions bend the straight line into curves**, and that bending is what lets neural networks learn complicated, nonlinear relationships.

There are a few standard activation functions. You do not need to memorize the math — just know what each one does and when to use it.

- **ReLU** (Rectified Linear Unit): returns the input if it is positive, otherwise zero. Simple, fast, and by far the most common choice for hidden layers.
- **Sigmoid**: squashes any number into the range $(0, 1)$. Used in the output layer for binary classification (gives a probability).
- **Tanh**: squashes any number into the range $(-1, 1)$. Similar shape to sigmoid but centered at zero. Sometimes used in hidden layers.
- **Softmax**: used in the output layer for multiclass classification. Turns several raw numbers into a probability distribution that sums to 1.



Key Takeaway — Choosing an Activation

Where	What to use
Hidden layers	ReLU (the default; works almost everywhere)
Binary output (yes/no)	Sigmoid
Multiclass output (A/B/C)	Softmax
Regression output (a number)	No activation (linear)

Under the Hood — Optional Formulas

For students who want to see the formulas:

- $\text{ReLU}(z) = \max(0, z)$
- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$
- Tanh: $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$
- Softmax turns each of K numbers z_k into a probability by dividing e^{z_k} by the sum of e^{z_j} across all K classes

You do not need to use these formulas by hand — scikit-learn computes them for you.

4 From One Neuron to One Layer

So far we have one neuron: many inputs in, one number out. A **layer** is just several neurons placed side by side, all looking at the same inputs. Each neuron has its own set of weights and its own bias, so each one learns to detect a different pattern.

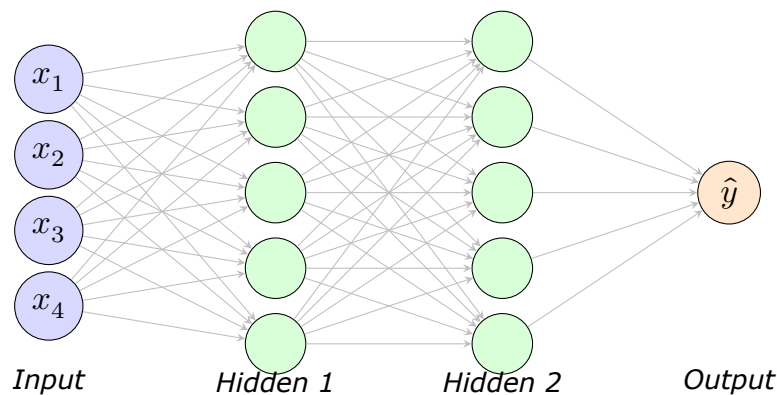
- **One neuron:** takes p inputs, produces 1 output
- **A layer with 5 neurons:** takes p inputs, produces 5 outputs (one from each neuron)
- **Important:** every neuron in the layer sees the exact same inputs. What makes them different is that they have different weights, so they react differently to the same data
- **Intuition:** think of a layer as a small team of experts, each looking at the same evidence but focusing on different clues

A layer with more neurons can spot more different patterns at once. The output of the whole layer is simply the collection of outputs from all its neurons, bundled together as a list of numbers that feeds into the next layer.

5 Stacking Layers: The Full Network

Now we stack layers on top of each other. The output of one layer becomes the input of the next, and the last layer produces the final prediction. This stack of layers is what people usually mean by a “neural network.”

The most common architecture for tabular business data is the **Multilayer Perceptron (MLP)** — a stack of fully connected layers where every neuron in one layer connects to every neuron in the next.



5.1 How a Prediction Flows Through the Network

Making a prediction with a trained network is called the **forward pass**. Think of it as information flowing left to right through the layers:

- **Input layer:** receives the raw features (e.g., tenure, monthly charges, contract type)

- **Hidden layer 1:** each neuron looks at the raw features, computes its weighted sum, applies ReLU, and outputs one number
- **Hidden layer 2:** each neuron now looks at the outputs from hidden layer 1 and does the same thing
- **More layers:** each hidden layer refines the representation from the previous layer
- **Output layer:** produces the final prediction (a probability for classification, or a number for regression)

At every layer, the neurons are doing the same simple thing you learned in the single-neuron section: weight the inputs, add bias, apply activation. The magic comes from stacking many of these simple computations so the network can learn complex, nonlinear patterns.

Key Takeaway — Why Stacking Helps

Early layers tend to learn simple patterns. Later layers combine those simple patterns into more abstract ones. For example, in an image network the first layer might detect edges, the second layer might detect shapes, the third layer might detect faces. This “hierarchy of features” is why deep networks are so powerful.

Try It in the Notebook

The companion notebook loads the IBM Telco Churn dataset, builds a multi-layer network, trains it end-to-end, and compares it against logistic regression and gradient boosting. That is the best way to see a forward pass happen on real business data.

6 Training: How the Network Learns from Mistakes

When a network is first created, its weights are random numbers. It will make terrible predictions. **Training** is the process of slowly adjusting those weights so the predictions get better.

The intuition is simple:

- **Step 1 — Make a guess.** Run some training data through the network (a forward pass) and see what it predicts
- **Step 2 — Measure how wrong it is.** Compare the prediction to the actual answer. The measure of “wrongness” is called the **loss**. A smaller loss means better predictions
- **Step 3 — Adjust the weights.** Nudge each weight slightly in the direction that would have reduced the loss. This nudging is done by an algorithm called **gradient descent**
- **Step 4 — Repeat.** Do steps 1–3 over and over, on many batches of training data. With each pass, the network gets a little better

One full pass through the entire training dataset is called an **epoch**. Networks usually train for many epochs until the loss stops improving.

6.1 What is the “Loss”?

The loss is just a score that says how wrong the network’s predictions are on the training data. Lower is better.

- **For classification** (e.g., will this customer churn?): the loss is a number that grows when the predicted probability is far from the true label. It is called **cross-entropy loss** or **log loss**
- **For regression** (e.g., what will this house sell for?): the loss is the average squared difference between the predicted number and the actual number. This is called **mean squared error (MSE)**

You do not have to compute loss by hand — scikit-learn and every deep learning library calculate it automatically.

6.2 The Intuition Behind Backpropagation

The algorithm that figures out “which weight should I nudge, and by how much” is called **backpropagation**. The name sounds scary, but the idea is not:

- **The output layer knows how wrong it was.** It can see the loss directly
- **It tells the layer behind it: “you contributed this much to my error, you should adjust by this much”**
- **That layer does the same to the layer behind it** — and so on, all the way back to the first hidden layer
- **Every neuron gets its own personalized update** based on how much it contributed to the final mistake
- **Think of it as a team passing blame backward**, with each member taking a fair share so everyone improves next time

Modern libraries handle backpropagation automatically — you never have to code it yourself.

Key Takeaway — The Training Loop

1. **Forward pass:** push inputs through the network, get a prediction
2. **Compute loss:** measure how wrong the prediction is
3. **Backward pass:** work out which weights contributed to the error and by how much
4. **Update weights:** nudge each weight a small step in the direction that reduces loss
5. Repeat for many batches over many epochs, until the loss stops improving

Under the Hood — Optional Loss Formulas

For students who want the formulas, the most common ones are:

- **Binary cross-entropy** (for yes/no problems): penalizes the network when it assigns a low probability to the correct class
- **Mean squared error** (for regression): average of $(\text{predicted} - \text{actual})^2$ across all training examples

Both are automatically computed by scikit-learn. You rarely need to see the raw formula.

7 The Adam Optimizer

In practice, nobody uses plain gradient descent anymore. They use a smarter variant called **Adam** (Adaptive Moment Estimation). Here is what Adam does differently:

- **Adapts the learning rate per weight.** Weights that change quickly get smaller updates; weights that change slowly get larger ones
- **Uses momentum.** Remembers the recent direction of updates so training moves smoothly instead of zig-zagging
- **Works well out of the box.** Rarely needs tuning
- **Is the default in virtually every modern deep learning library**, including scikit-learn (`MLPClassifier(solver="adam")`)

For almost every business problem, “use Adam” is the right answer.

8 Regularization: Keeping the Network Honest

Neural networks have a lot of weights, which means they can easily memorize the training data instead of learning the real pattern. That is called **overfitting**, and it makes test-set performance terrible. Three standard techniques keep the network honest.

8.1 L2 Regularization (Weight Decay)

- **What it does:** adds a small penalty for large weights to the loss
- **Why it helps:** big weights let the network memorize, so penalizing them forces it to keep weights small and generalize better
- **In scikit-learn:** controlled by the `alpha` parameter of `MLPClassifier` (higher `alpha` = stronger regularization)

8.2 Dropout

- **What it does:** during training, randomly “turns off” a fraction of the hidden neurons on each forward pass
- **Why it helps:** the network cannot rely on any single neuron, so it learns more robust, distributed patterns
- **Only during training:** at prediction time, all neurons are active again

8.3 Early Stopping

- **What it does:** watches the validation loss during training and stops as soon as it stops improving
- **Why it helps:** prevents the network from continuing to train past the point where it starts overfitting
- **In scikit-learn:**

```
mlp = MLPClassifier(early_stopping=True,
                    validation_fraction=0.1,
                    n_iter_no_change=10)
```

Important Note — Dropout in scikit-learn

Scikit-learn’s `MLPClassifier` supports L2 regularization and early stopping but does NOT natively support dropout. If you need dropout, use PyTorch, TensorFlow, or Keras. For the tabular business problems in this course, L2 and early stopping are usually enough.

9 Building a Neural Network in Python

Here is the minimal skeleton for training an MLP classifier in scikit-learn. Do not expect to understand every argument yet — just get a feel for how little code it takes.

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

# ALWAYS scale features first
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32), # two hidden layers
    activation="relu",
    solver="adam",
    early_stopping=True,
```

```

random_state=42
)
mlp.fit(X_train_s, y_train)

```

Try It in the Notebook

The companion notebook walks through the full workflow end-to-end on the IBM Telco Churn dataset:

- Loading and scaling the data
- Training an MLP and comparing it against logistic regression and gradient boosting
- Showing what happens when you forget to scale (spoiler: it breaks)
- Tuning hyperparameters with `GridSearchCV`

Do the hands-on work there — that is where the real learning happens.

Common Mistake — Not Scaling Features

Neural networks assume features are on comparable scales. Without scaling, a feature like income (thousands of dollars) will dominate a feature like age (tens of years), and training will fail or converge extremely slowly. **Always scale before training an MLP.**

10 Choosing Your Architecture

There is no universal recipe for how many layers and neurons to use. The following rules of thumb cover most business problems:

- **Start small.** One hidden layer with 32–128 neurons is usually enough for tabular data. Do not start with a 10-layer network
- **Widen before you deepen.** For tabular data, adding more neurons to a single layer usually helps more than adding a new layer
- **Treat architecture as a hyperparameter.** Use cross-validation to pick the size, just like you pick `max_depth` for a decision tree
- **More data means you can go bigger.** Deep networks really shine when you have millions of examples. With only a few thousand rows, a shallow network is fine
- **Stick with ReLU** in the hidden layers unless you have a specific reason not to

Key Takeaway — Neural Networks on Tabular Data

For small-to-medium **tabular** business datasets, gradient boosting (XGBoost, LightGBM) usually beats a neural network — with less tuning and faster training. Neural networks dominate on:

- **Images:** convolutional neural networks (CNNs)
- **Text and language:** transformers (BERT, GPT, Claude)
- **Speech and audio**
- **Very large tabular datasets** with millions of rows and complex interactions

For most business problems in this course, reach for an ensemble first. Use a neural network when the data is unstructured (text or images) or enormous.

Practice — When Does a ReLU Neuron Fire?

Suppose a hidden neuron in a network uses ReLU as its activation function.

- The neuron's weighted sum plus bias comes out to -0.5 . What does the neuron output?
- On a different input, the weighted sum plus bias comes out to 2.3 . What does the neuron output?
- In your own words, what does it mean for a ReLU neuron to "fire" versus to "stay silent"? Why do you think this property is useful for a neural network?

Answer

- ReLU turns any negative number into 0, so the neuron outputs **0**. It stays silent
- ReLU leaves positive numbers alone, so the neuron outputs **2.3**. It fires with a strength of 2.3
- A ReLU neuron "fires" (outputs a positive number) when its weighted sum plus bias is positive, and "stays silent" (outputs 0) otherwise. This is useful because different neurons end up firing on different patterns in the data — one neuron might fire on customers with long tenure, another on customers with high monthly charges — and the network learns which combinations matter. It also speeds up training because silent neurons contribute nothing to the next layer, effectively simplifying the computation

11 Advantages and Limitations

11.1 Advantages

- Can learn highly nonlinear relationships
- Scale with data — more examples usually mean better performance
- State-of-the-art on images, text, and audio
- Same framework handles classification and regression

11.2 Limitations

- **Data hungry:** need a lot of examples to shine
- **Hard to interpret:** the “black box” reputation is largely deserved
- **Slow to train:** much slower than trees or linear models
- **Many hyperparameters:** architecture, learning rate, batch size, regularization
- **Sensitive to scaling:** always standardize inputs
- **Nondeterministic:** different random seeds produce different results

12 From Neural Networks to Large Language Models

ChatGPT, Claude, Gemini, and Llama are some of the most widely used AI products in the world. Every single one of them is a neural network. They use the exact same building blocks you just learned — layers of neurons, weights, biases, activation functions, loss, gradient descent, backpropagation — just at a dramatically larger scale and with a specialized design called the **Transformer**.

Key Takeaway — LLMs Are Still Just Neural Networks

The building blocks are the same:

- Neurons compute weighted sums and pass them through activation functions
- Training uses gradient descent (usually Adam) and backpropagation
- Regularization uses techniques like dropout, weight decay, and early stopping

What changes at the LLM scale:

- **Architecture:** Transformer (not MLP or CNN) to handle sequences of words
- **Size:** hundreds of billions to trillions of weights, not thousands
- **Data:** trained on hundreds of billions of words of text
- **Compute:** thousands of GPUs running for weeks or months per training run

12.1 The Neural Network Family Tree

Different problems call for different neural network architectures:

MLP

Best for: Tabular data
Fully connected layers.
What we studied above.
Example: Churn, credit scoring.

CNN

Best for: Images
Convolutional layers extract spatial patterns like edges and shapes.
Example: Image classification, medical scans.

Transformer

Best for: Sequences (text, code, audio)
Attention layers learn relationships between elements in a sequence.
Example: ChatGPT, machine translation, search.

All three are neural networks trained with backpropagation. They differ mainly in **how the neurons are connected**, which reflects the structure of the problem they are meant to solve.

12.2 How Transformers Work: Four Key Ideas

12.2.1 1. Tokens and Embeddings

An LLM does not see raw text. Before anything happens, the input is chopped into small chunks called **tokens** — roughly 1–4 characters each. Each token is then mapped to a long list of numbers called an **embedding**, which captures the token's meaning.

- A token is like a puzzle piece of text (roughly a word or part of a word)
- An embedding is a list of several thousand numbers that represents the token's meaning
- These embeddings are learned from data — nothing is hand-coded
- The sentence "Predictive Analytics is fun" becomes about 5 tokens, each replaced by a long list of numbers that the network can work with

12.2.2 2. Self-Attention

The defining idea of the Transformer is **self-attention**. For each word in a sentence, self-attention lets that word "look at" every other word and figure out which ones are important for understanding it.

Key Takeaway — What Attention Does

Attention lets each word in a sentence look at every other word and decide how much to rely on it for context. In the sentence "*The customer called the bank because she was upset about her fee,*" the word "*she*" attends heavily to "*customer*" (its antecedent) and very little to "*bank*" because the bank is not who was upset. The model learns these attention patterns from reading billions of sentences during training.

- Each token asks every other token: "how relevant are you to me?"
- Tokens that are highly relevant get a larger weight in the current token's updated representation
- Tokens that are irrelevant get almost zero weight
- This happens for every token in the sentence, at every layer, in parallel

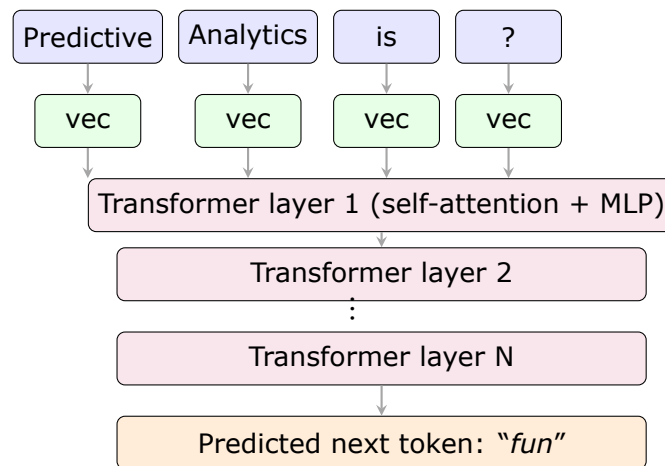
12.2.3 3. Stacked Layers

A Transformer is a stack of many attention layers on top of each other (GPT-3 has 96 layers; GPT-4 has over 100). Each layer refines every token's representation using the same self-attention mechanism, followed by a small MLP. After all the layers, each token has absorbed context from every other token in the sequence.

12.2.4 4. Next-Token Prediction

LLMs are trained on a disarmingly simple goal: **given the words so far, predict the next word.**

- The training data is raw text from the internet, books, research papers, and code
- The model outputs a probability for every possible next token (roughly 50,000 options)
- The loss is how far off the predicted probabilities are from the true next token
- At inference time, the model generates one token at a time, feeding each new token back into the input and predicting the next one
- That is how ChatGPT writes a paragraph: one token, then the next, then the next



12.3 Scale Changes Everything

The Transformer paper (Vaswani et al., 2017) described a model with about 65 million weights. By 2023, frontier LLMs had grown by four orders of magnitude:

Model	Approx. Number of Weights	Year
Original Transformer	65 million	2017
GPT-2	1.5 billion	2019
GPT-3	175 billion	2020
GPT-4 / Claude / Gemini	over 1 trillion (estimated)	2023–2024

As scale grew, new capabilities suddenly appeared — few-shot learning, step-by-step reasoning, code generation — that smaller models simply could not do. This is called **emergence**, and it is one of the most debated phenomena in modern AI.

12.4 What LLMs Are and Aren't

Important Note — Pattern Matching, Not Understanding

LLMs are trained to predict the next token. They are extremely good at it, which produces fluent and often helpful output — but under the hood they are **statistical pattern matchers**, not reasoning engines. Specifically:

- They can “hallucinate” facts that sound plausible but are wrong
- They have no inherent access to information beyond their training cutoff
- They cannot independently verify their own output
- They inherit biases that exist in their training data

For serious business use, always pair an LLM's output with verification, grounding, or retrieval from trusted sources whenever correctness matters.

12.5 Business Applications of LLMs

Where traditional neural networks and gradient boosting dominate tabular data, LLMs dominate tasks involving **unstructured text**:

Customer Support

Chatbots that answer routine questions, triage tickets, and escalate complex issues to humans.

Document Summarization

Condensing long contracts, reports, and research into bullet points or executive summaries.

Code Assistance

GitHub Copilot and similar tools help developers write and debug code faster.

Semantic Search

Searching documents by meaning rather than exact keyword matches, using text embeddings.

Key Takeaway — Choosing the Right Tool

Problem type

Best tool

Structured tabular data

Gradient boosting (XGBoost, LightGBM)

Images

CNN or vision transformer

Text classification, small dataset

Logistic regression on text embeddings

Text generation, summarization, Q&A

LLM (Claude, GPT, Gemini)

Mixed tabular + text features

Gradient boosting on tabular + LLM-derived text embeddings

LLMs are remarkable, but they are not the right tool for every problem. For predictive analytics on business tabular data, a well-tuned gradient boosting model will usually beat an LLM while being orders of magnitude cheaper and more interpretable.

13 Summary

What We Covered

Topic	Key Concepts
Single neuron	Inputs, weights, bias, activation, output
Activation functions	ReLU (hidden), sigmoid/softmax (output)
From one neuron to a layer	Many neurons looking at the same inputs in parallel
Stacking layers	Each layer's outputs become the next layer's inputs
Training (learning from mistakes)	Forward pass, loss, backpropagation, weight updates
Adam optimizer	Smart version of gradient descent; default in practice
Regularization	L2, dropout, early stopping
Architecture	Start small; wider before deeper for tabular
LLMs & Transformers	Same building blocks, new connection pattern (attention), massive scale
Next week: Unsupervised learning — clustering and segmentation when there is no labeled target variable.	